

MDC-102 Flutter：Material Design 結構和版面配置

來源：<https://codelabs.developers.google.com/codelabs/mdc-102-flutter?hl=zh-tw>

步驟數：7

瞭解如何使用 Material Design 建立 Flutter 應用程式結構和版面配置。

目錄

1. [1. 簡介](#)
2. [2. 設定 Flutter 開發環境](#)
3. [3. 下載程式碼實驗室範例應用程式](#)
4. [4. 新增頂端應用程式列](#)
5. [5. 在格線中新增資訊卡](#)
6. [6. 製作資訊卡集錦](#)
7. [7. 恭喜！](#)

1. 簡介



開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

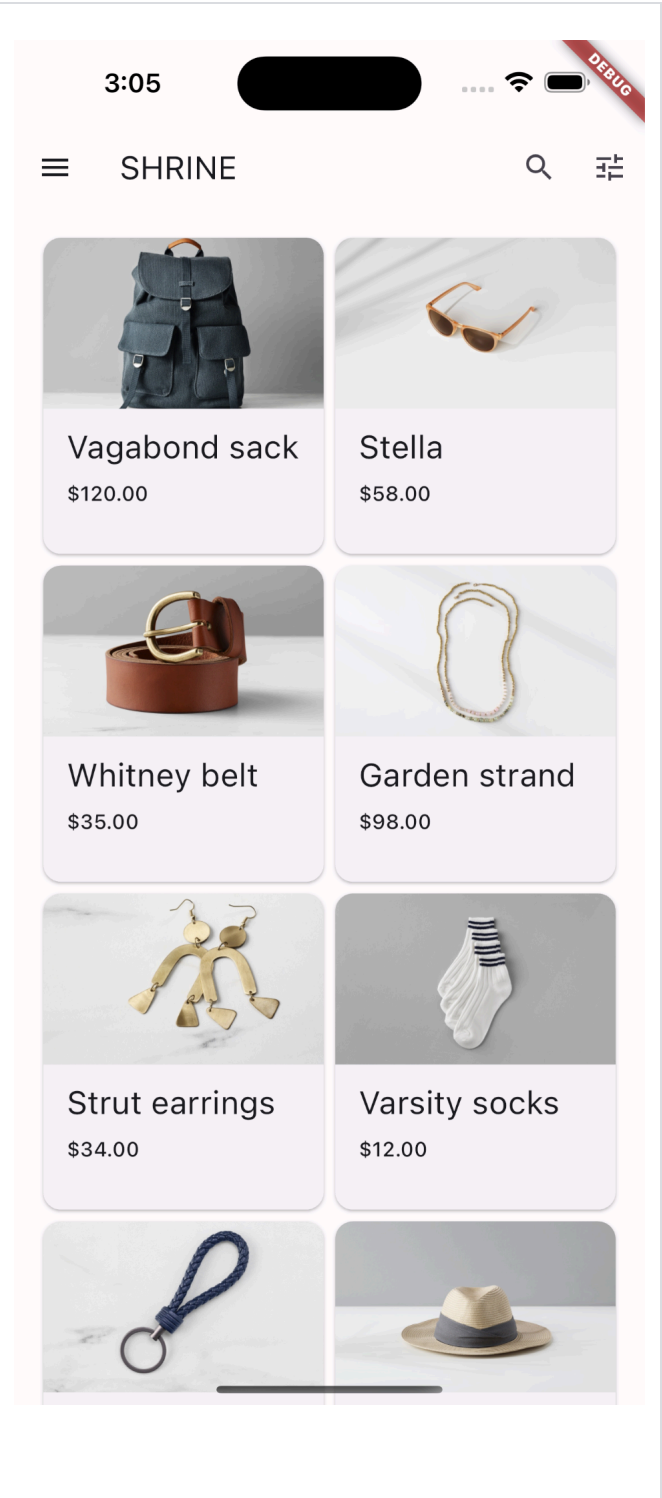
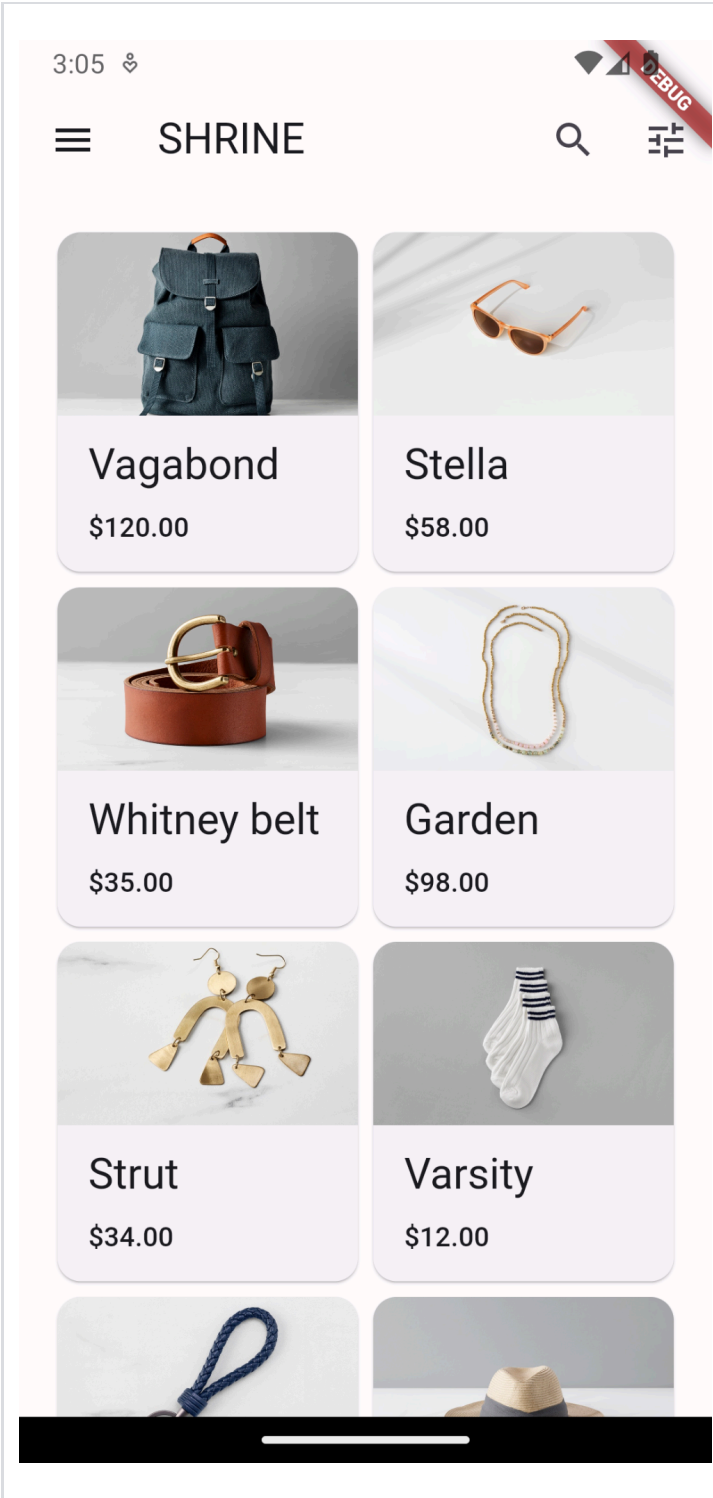
在 [MDC-101](#) 程式碼研究室中，您使用了兩個 Material Design 元件來建構登入頁面：文字欄位和帶有墨水漣漪效果的按鈕。現在，我們將新增導覽、結構和資料，擴充這個基礎。

建構項目

在本程式碼研究室中，您將為名為「**Shrine**」的應用程式建構主畫面。這款電子商務應用程式販售服飾和居家用品。其中包含：

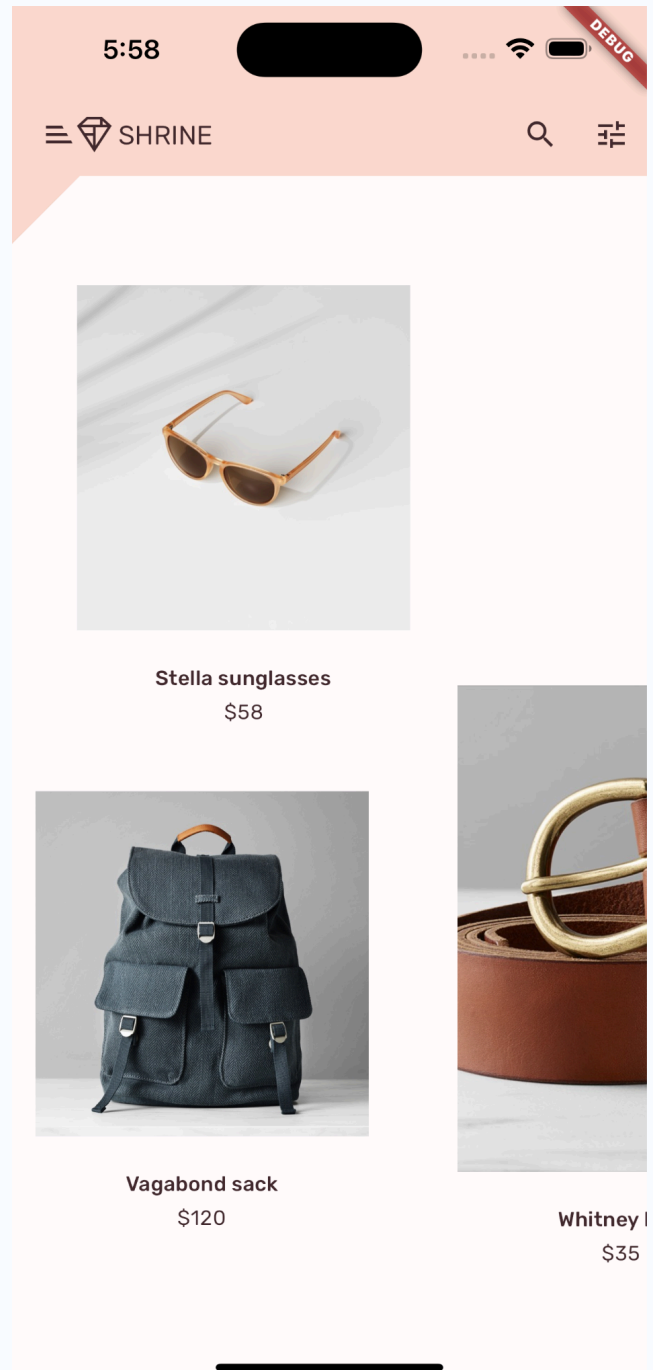
- 頂端應用程式列
- 滿滿的產品格狀清單

Android	iOS
---------	-----



這是四個程式碼研究室的第 2 個，可引導您為名為 Shrine 的產品建構應用程式。建議您依序完成所有程式碼研究室，逐步完成各項工作。相關程式碼研究室如下：

- [MDC-101 Flutter：Material Design 元件基本概念](#)
- [MDC-103 Flutter：利用顏色、形狀、高度和類型建立 Material Design 主題](#)
- [MDC-104 Flutter：Material 進階元件](#)



本程式碼研究室中的 Material Flutter 元件和子系統

- 頂端應用程式列
- 格線
- 資訊卡

在本程式碼研究室中，您將使用 Material Flutter 提供的預設元件。您將在 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，瞭解如何自訂這些屬性。

2. 設定 Flutter 開發環境

如要完成本實驗室，您需要兩項軟體：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室：

- 連線至電腦並設為開發人員模式的實體 [Android](#) 或 [iOS](#) 裝置。
 - [iOS 模擬器](#) (需要安裝 Xcode 工具)。
 - [Android Emulator](#) (需在 Android Studio 中設定)。
 - 瀏覽器 (偵錯時必須使用 Chrome)。
 - 以 [Windows](#)、[Linux](#) 或 [macOS](#) 電腦版應用程式的形式。您必須在要部署的平台上進行開發。因此，如要開發 Windows 桌面應用程式，您必須在 Windows 上開發，才能存取適當的建構鏈。如需作業系統專屬需求，請參閱 docs.flutter.dev/desktop。
-

步驟 3 · 約 0 分鐘

3. 下載程式碼實驗室範例應用程式

是否要從 MDC-101 繼續？

如果您已完成 MDC-101，程式碼應該已準備好用於本程式碼研究室。跳至「[加入頂端應用程式列](#)」步驟。

從零開始？

下載程式碼研究室範例應用程式

範例應用程式位於 `material-components-flutter-codelabs-102-starter_and_101-complete/mdc_100_series` 目錄中。

...或從 GitHub 複製

如要從 GitHub 複製本程式碼研究室，請執行下列指令：

```
git clone https://github.com/material-components/material-components-flutter-codelabs.git
cd material-components-flutter-codelabs/mdc_100_series
git checkout 102-starter_and_101-complete
```

如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

正確的分支版本

MDC-101 到 104 程式碼研究室會循序漸進，因此，當您完成 102 的程式碼後，該程式碼就會成為 103 的範例程式碼！程式碼會分散在不同分支中，您可以使用下列指令列出所有分支：

```
git branch --list
```

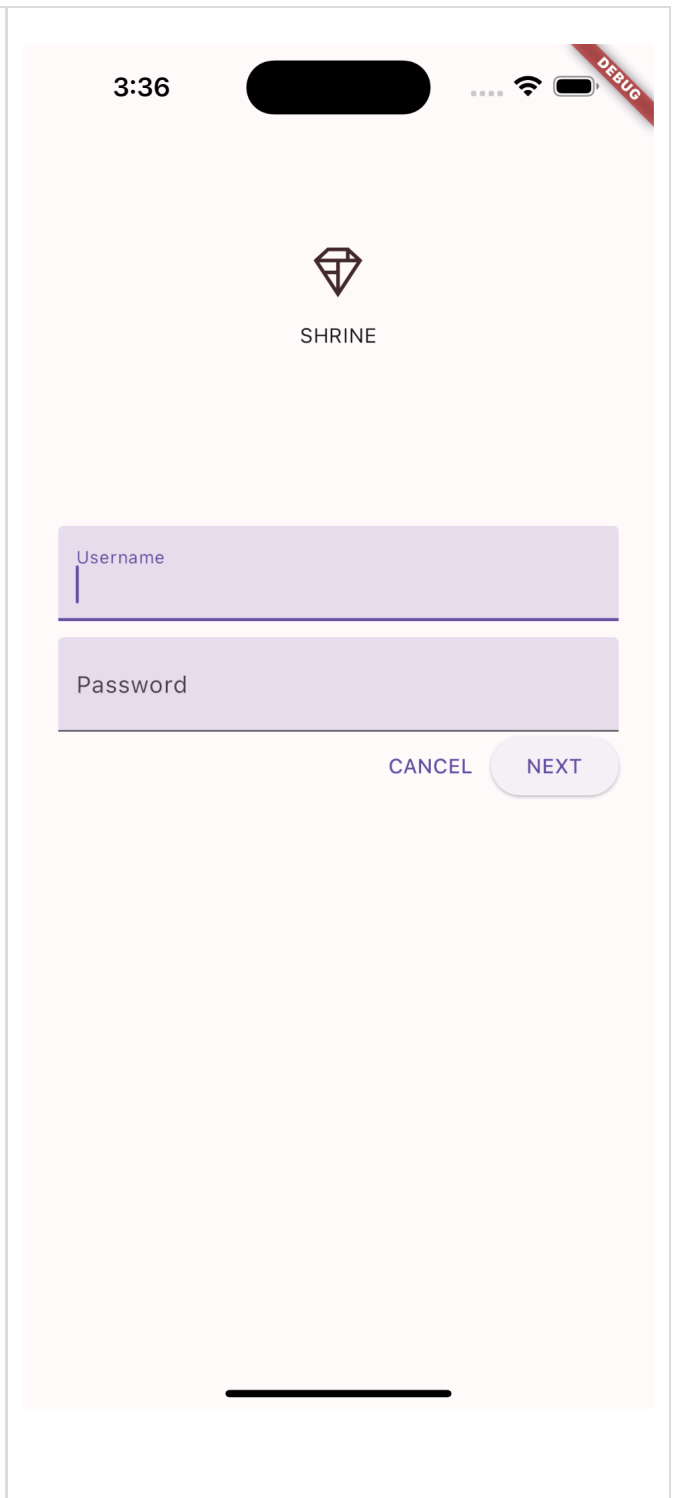
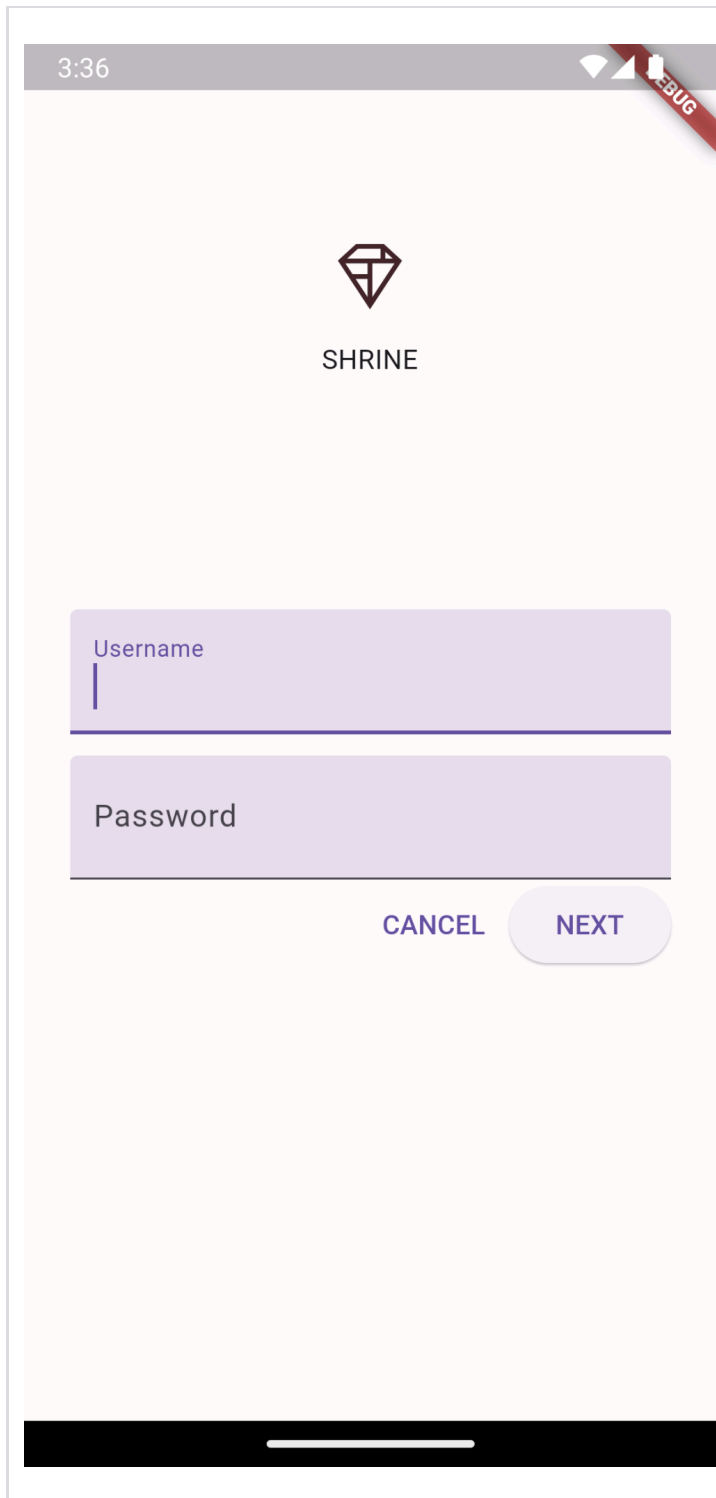
如要查看完成的程式碼，請切換至 `103-starter_and_102-complete` 分支。

開啟專案並執行應用程式

1. 在您選擇的編輯器中開啟專案。
2. 按照所選編輯器的「[開始使用：試用](#)」一節中的「執行應用程式」操作說明操作。

太棒了，裝置上應會顯示 MDC-101 程式碼研究室的 Shrine 登入頁面。

Android	iOS
---------	-----



登入畫面現在看起來不錯，接下來請在應用程式中填入一些產品。

4. 新增頂端應用程式列

現在點選「繼續」按鈕，即可看到顯示「完成！」的主畫面。太好了！但現在使用者無法採取任何動作，也無法瞭解自己在應用程式中的位置。為此，我們需要加入導覽功能。

導覽是指可讓使用者在應用程式中移動的元件、互動、視覺提示和資訊架構。導覽可協助使用者發掘內容和功能，輕鬆完成工作。

詳情請參閱 Material Guidelines 中的[導覽文章](#)。

Material Design 提供導覽模式，確保高度可用性。頂端應用程式列就是其中一個最顯眼的元件。

頂端應用程式列在 iOS 中可能稱為「導覽列」，或簡稱為「應用程式列」或「標題」。

為了提供導覽功能，並讓使用者快速存取其他動作，請新增頂端應用程式列。

新增 AppBar 小工具

在 `home.dart` 中，將 AppBar 新增至 Scaffold，並移除醒目顯示的 `const`：

```
return const Scaffold(  
  // TODO: Add app bar (102)  
  appBar: AppBar(  
    // TODO: Add buttons and title (102)  
  ),  
);
```

將 **AppBar** 新增至 Scaffold 的 `appBar:` 欄位，即可免費取得完美版面配置，將 AppBar 放在頁面頂端，主體則放在下方。

Scaffold 是 MaterialApps 的重要小工具。並提供便利的 API，可顯示各種或常見的 Material 元件，例如抽屜、SnackBar 和底部功能表。甚至還能協助配置懸浮動作按鈕。

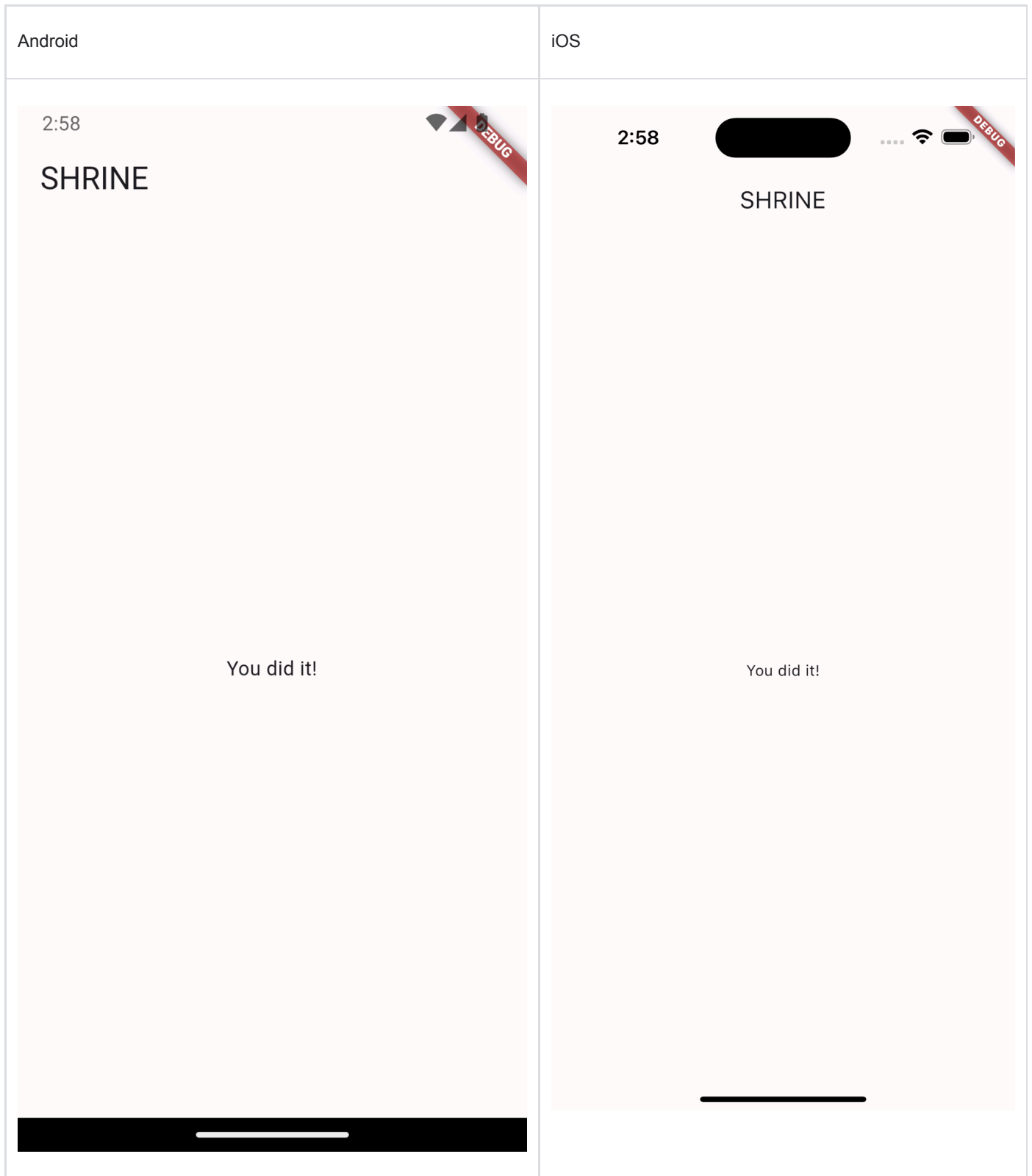
如要進一步瞭解 Scaffold，請參閱 [Flutter 說明文件](#)。

新增文字小工具

在 `home.dart` 中，為 AppBar 新增標題：

```
// TODO: Add app bar (102)  
appBar: AppBar(  
  // TODO: Add buttons and title (102)  
  title: const Text('SHRINE'),  
  // TODO: Add trailing buttons (102)  
);
```

儲存專案。



到目前為止，您可能已注意到所謂的「平台差異」。Material 知道 Android、iOS 和網頁等平台各不相同，而且使用者期望不同。舉例來說，在 iOS 上，標題幾乎一律會置中，這是 UIKit 提供的預設行為。在 Android 裝置上，文字會靠左對齊。因此，如果您使用 Android 模擬器或裝置，標題應靠左對齊。如果是 iOS 模擬器或裝置，則應置中。

詳情請參閱 [Material 文章](#)，瞭解跨平台適應性。

許多應用程式列的標題旁都有按鈕。我們在應用程式中新增選單圖示。

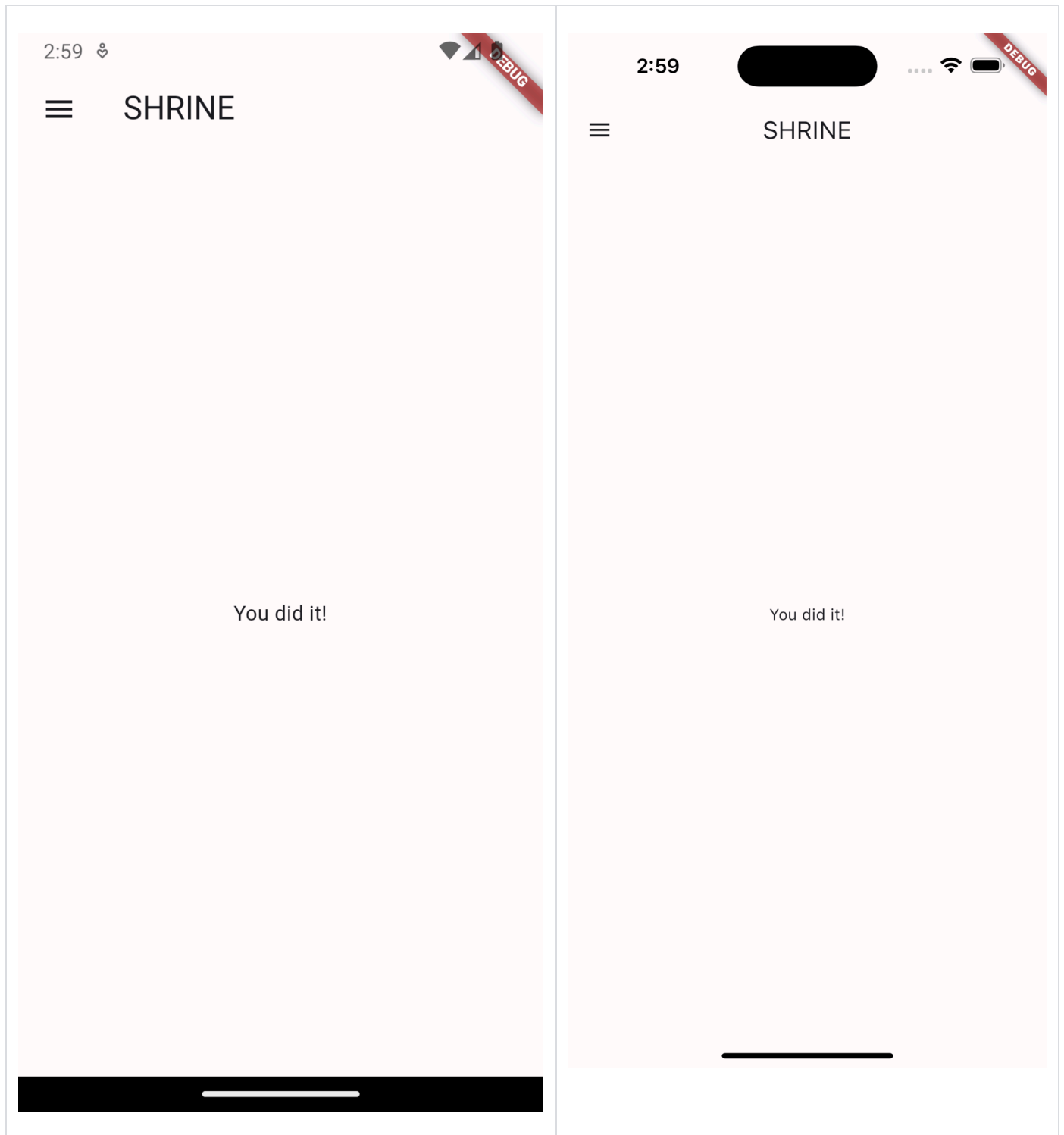
新增開頭的 IconButton

在 `home.dart` 中，為 AppBar 的 `leading:` 欄位設定 `IconButton`。(請將其放在 `title:` 欄位前面，模擬從頭到尾的順序)：

```
// TODO: Add buttons and title (102)
leading: IconButton(
  icon: const Icon(
    Icons.menu,
    semanticLabel: 'menu',
  ),
  onPressed: () {
    print('Menu button');
  },
),
```

儲存專案。

Android	iOS
---------	-----



選單圖示 (又稱「漢堡」) 會顯示在您預期的位置。

IconButton 類別可讓您輕鬆在應用程式中加入 [Material Design 圖示](#)。這個類別會採用 **Icon** 小工具。Flutter 的 **Icons** 類別中有一整套圖示。系統會根據常數字串對應自動匯入圖示。

如要進一步瞭解 Icons 類別，請參閱 [Flutter 說明文件](#)。如要進一步瞭解 Icon 小工具，請參閱 [Flutter 說明文件](#)。

您也可以在標題的尾端新增按鈕。在 Flutter 中，這些項目稱為「動作」。

「前導」和「尾隨」是表示方向的用語，以不限語言的方式指稱文字行的開頭和結尾。如果使用由左至右 (LTR) 的語言 (例如英文)，「leading」表示「left」，「trailing」表示「right」。在阿拉伯文等 RTL (由右至左) 語言中，「leading」是指「right」，「trailing」是指「left」。

如要進一步瞭解 UI 鏡像，請參閱「[雙向性](#)」的 Material Design 指南。

新增動作

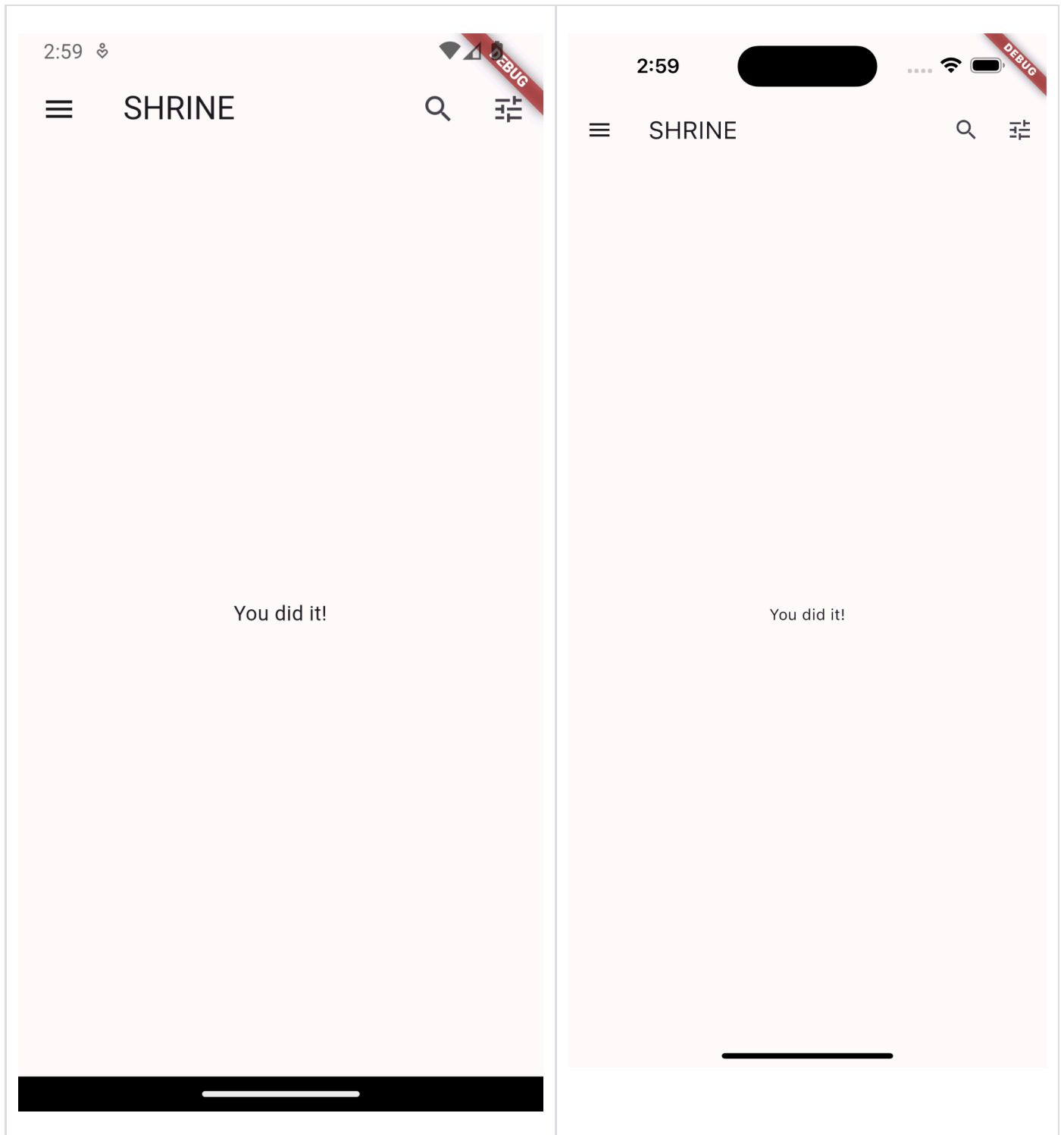
還可容納兩個 `IconButton`。

在標題後方將這些項目新增至 `AppBar` 執行個體：

```
// TODO: Add trailing buttons (102)
actions: <Widget>[
  IconButton(
    icon: const Icon(
      Icons.search,
      semanticLabel: 'search',
    ),
    onPressed: () {
      print('Search button');
    },
  ),
  IconButton(
    icon: const Icon(
      Icons.tune,
      semanticLabel: 'filter',
    ),
    onPressed: () {
      print('Filter button');
    },
  ),
],
```

儲存專案。主畫面應如下所示：

Android	iOS
---------	-----



現在應用程式有開頭按鈕、標題，以及右側的兩個動作。應用程式列也會使用細微的陰影顯示高度，表示應用程式列位於與內容不同的圖層。

在 `Icon` 類別中，**`SemanticLabel`** 欄位是 Flutter 中新增無障礙資訊的常見方式。這與 [Android 的內容標籤](#) 和 [iOS 的 `UIAccessibility` `accessibilityLabel`](#) 非常相似。許多課程都會提供這項功能。

這個欄位中的資訊可協助使用螢幕閱讀器的人員瞭解按鈕功能

如果 `Widget` 沒有 `semanticLabel` 欄位，可以將其包裝在 **`Semantics`** `Widget` 中。如要進一步瞭解語意小工具，請參閱 [Flutter 說明文件](#)。

5. 在格線中新增資訊卡

現在應用程式已具備一些結構，接下來請將內容放入資訊卡，以便整理。

資訊卡是獨立元素，可顯示單一主題的內容和動作。這類廣告可彈性呈現類似內容的集合。

若要進一步瞭解資訊卡，請參閱 Material 指南的[資訊卡文章](#)。

若要進一步瞭解 Card 小工具，請參閱「[在 Flutter 中建構版面配置](#)」。

新增 GridView

首先，在頂端應用程式列下方新增一張資訊卡。單獨的 **Card** 小工具沒有足夠的資訊可供版面配置，因此我們會將其封裝在 **GridView** 小工具中。

將 Scaffold 主體中的 Center 替換為 GridView：

```
// TODO: Add a grid view (102)
body: GridView.count(
  crossAxisCount: 2,
  padding: const EdgeInsets.all(16.0),
  childAspectRatio: 8.0 / 9.0,
  // TODO: Build a grid of cards (102)
  children: <Widget>[Card()],
),
```

讓我們來解讀這段程式碼。GridView 會叫用 `count()` 建構函式，因為顯示的項目數量可數，並非無限。但需要更多資訊才能定義版面配置。

`crossAxisCount`：會指定橫向的項目數量。我們需要 2 欄。

在 Flutter 中，「交叉軸」是指非捲動軸。捲動方向稱為主軸。因此，如果您採用垂直捲動 (例如 GridView 預設的捲動方式)，則交叉軸為水平軸。

詳情請參閱「[建構版面配置](#)」。

`padding`：欄位會在 GridView 的 4 個側邊提供空間。當然，您無法看到尾端或底部的邊框間距，因為旁邊還沒有 GridView 子項。

`childAspectRatio`：欄位會根據長寬比 (寬度/高度) 識別項目大小。

根據預設，GridView 會製作大小相同的圖塊。

我們有一張資訊卡，但內容空白。讓我們在資訊卡中新增子項小工具。

排版內容

資訊卡應包含圖片、標題和次要文字的區域。

更新 GridView 的子項：

```
// TODO: Build a grid of cards (102)
children: <Widget>[
  Card(
    clipBehavior: Clip.antiAlias,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        AspectRatio(
          aspectRatio: 18.0 / 11.0,
          child: Image.asset('assets/diamond.png'),
        ),
        Padding(
          padding: const EdgeInsets.fromLTRB(16.0, 12.0, 16.0, 8.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              Text('Title'),
              const SizedBox(height: 8.0),
              Text('Secondary Text'),
            ],
          ),
        ),
      ],
    ),
  ],
),
],
```

這段程式碼會新增 **Column** 小工具，用於垂直排列子項小工具。

`crossAxisAlignment: field` 指定 `CrossAxisAlignment.start`，表示「將文字對齊前緣」。

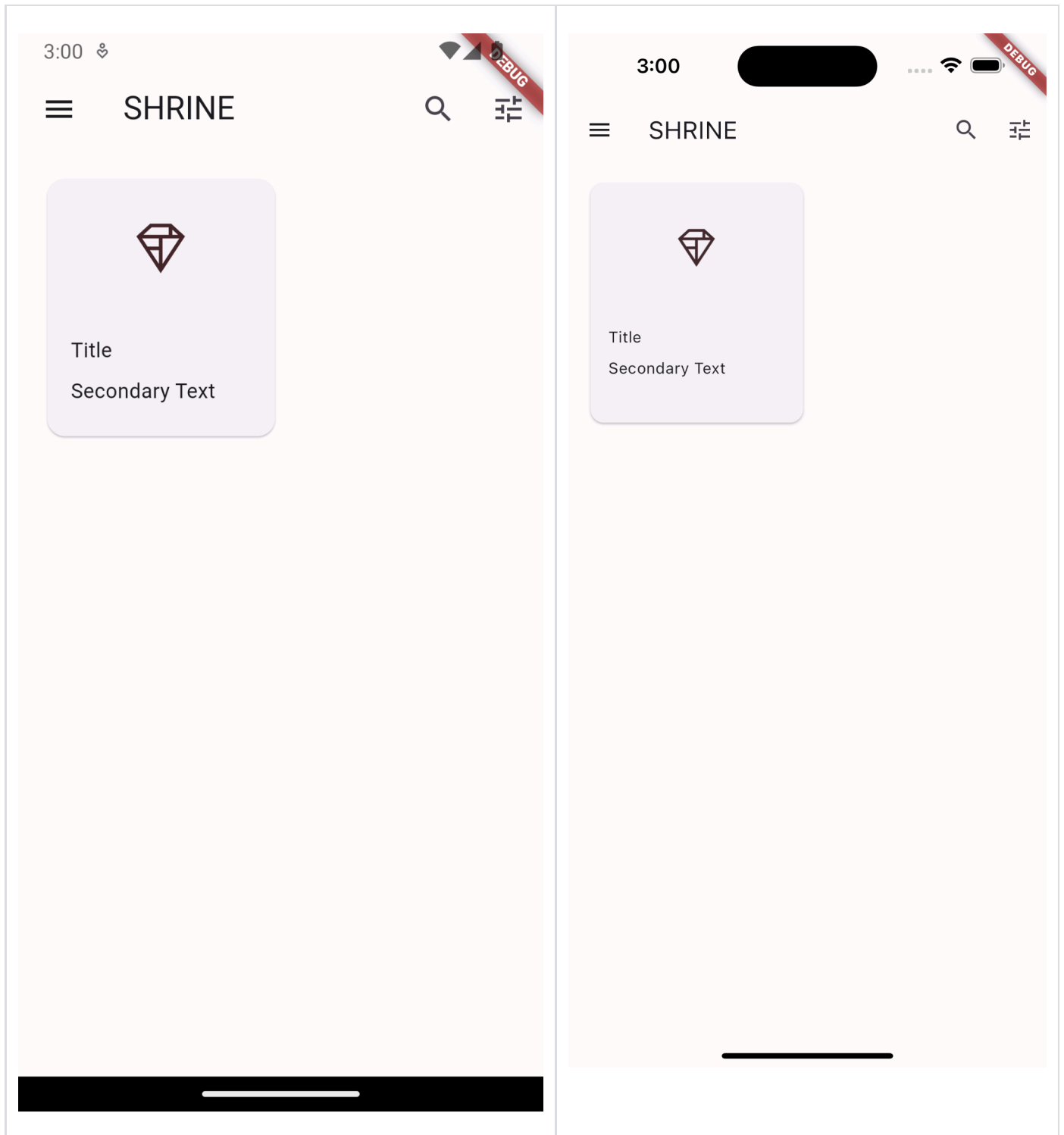
無論提供哪種圖片，**AspectRatio** 小工具都會決定圖片的形狀。

邊框間距會將文字稍微往內移。

這兩個 **Text** 小工具會垂直堆疊，中間有 8 點的空白空間 (**SizedBox**)。我們再建立一個 **Column**，將這些項目放在 **Padding** 內。

儲存專案。

Android	iOS
---------	-----



在預覽畫面中，您可以看到資訊卡從邊緣內縮，並有圓角和陰影 (表示資訊卡的高度)。在 Material 中，整個形狀稱為「容器」。(請勿與名為 [Container](#) 的實際小工具類別混淆)。

除了容器之外，Material 中的所有元素實際上都是選用項目。你可以新增標題文字、縮圖或顯示圖片、子標題文字、分隔線，甚至是按鈕和圖示。

如要進一步瞭解資訊卡的內容，請參閱 Material 指南的[資訊卡文章](#)。

資訊卡通常會與其他資訊卡一起顯示在集合中。我們將這些圖片以格狀集合的形式呈現。

6. 製作資訊卡集錦

如果畫面上有多張資訊卡，系統會將這些資訊卡分組為一或多個集合。集合中的資訊卡共平面，也就是說，資訊卡彼此之間共用相同的靜止高度 (除非資訊卡遭到選取或拖曳，但我們不會在這裡執行這項操作)。

將資訊卡複製到收藏

目前我們的 Card 是在 GridView 的 `children` 欄位中建構。這會產生大量巢狀程式碼，可能難以閱讀。讓我們將其擷取至函式，盡可能產生空白資訊卡，並傳回資訊卡清單。

在 `build()` 函式上方建立新的私有函式 (請注意，開頭為底線的函式是私有 API)：

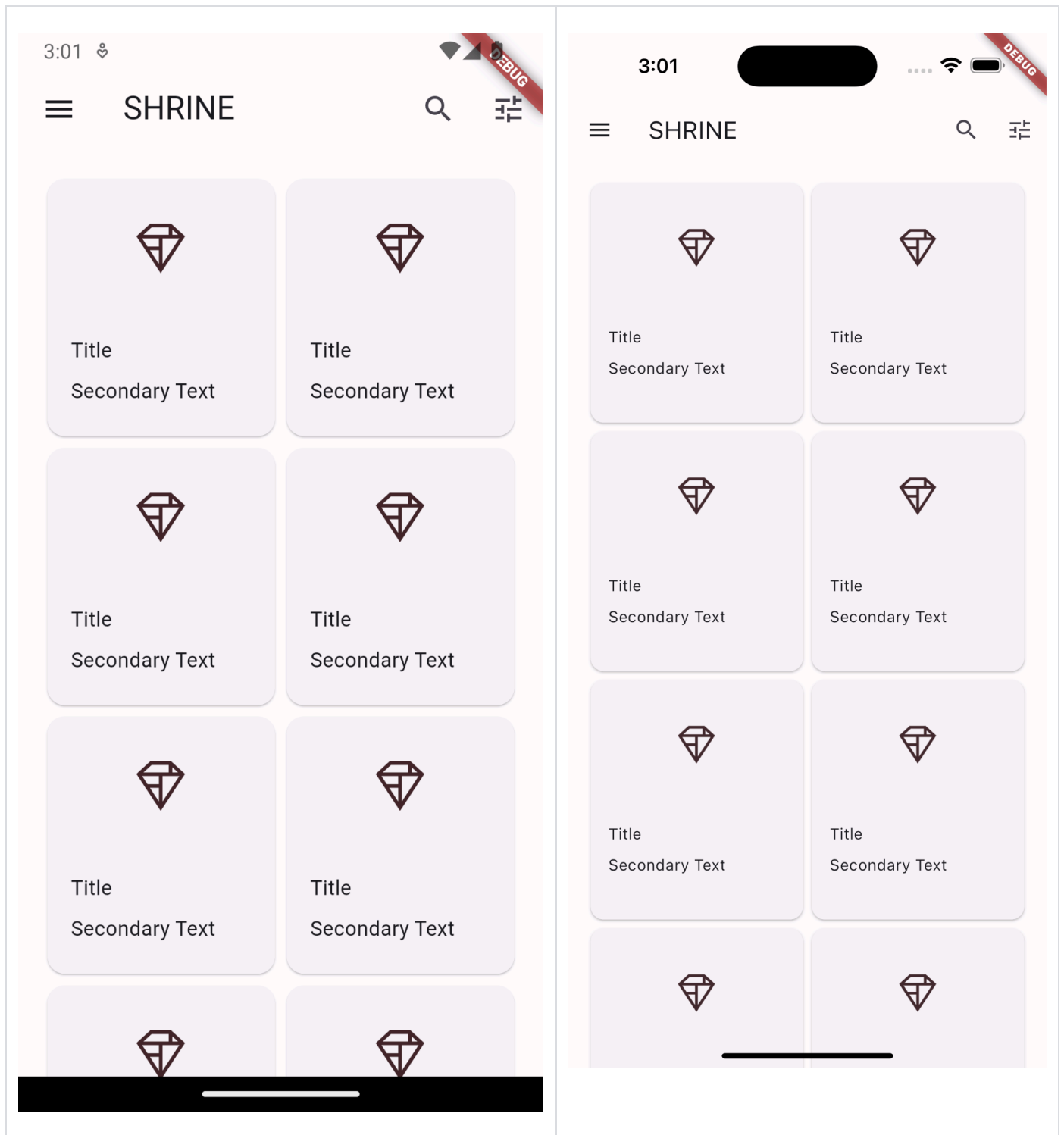
```
// TODO: Make a collection of cards (102)
List<Card> _buildGridCards(int count) {
  List<Card> cards = List.generate(
    count,
    (int index) {
      return Card(
        clipBehavior: Clip.antiAlias,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            AspectRatio(
              aspectRatio: 18.0 / 11.0,
              child: Image.asset('assets/diamond.png'),
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(16.0, 12.0, 16.0, 8.0),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: const <Widget>[
                  Text('Title'),
                  SizedBox(height: 8.0),
                  Text('Secondary Text'),
                ],
              ),
            ),
          ],
        ),
      );
    },
  );
  return cards;
}
```

將產生的資訊卡指派給 GridView 的 `children` 欄位。請記得使用這段新程式碼取代 **GridView** 中的所有內容：

```
// TODO: Add a grid view (102)
body: GridView.count(
  crossAxisCount: 2,
  padding: const EdgeInsets.all(16.0),
  childAspectRatio: 8.0 / 9.0,
  children: _buildGridCards(10) // Replace
),
```

儲存專案。

Android	iOS
---------	-----



資訊卡已顯示，但目前沒有任何內容。現在可以新增產品資料了。

新增產品資料

應用程式中有一些產品，並附上圖片、名稱和價格。讓我們將該項目新增至資訊卡中現有的小工具

接著，在 `home.dart` 中匯入新套件，以及我們為資料模型提供的部分檔案：

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';

import 'model/product.dart';
import 'model/products_repository.dart';
```

最後，請變更 `_buildGridCards()` 以擷取產品資訊，並在資訊卡中使用該資料：

```

// TODO: Make a collection of cards (102)

// Replace this entire method
List<Card> _buildGridCards(BuildContext context) {
  List<Product> products = ProductsRepository.loadProducts(Category.all);

  if (products.isEmpty) {
    return const <Card>[];
  }

  final ThemeData theme = Theme.of(context);
  final NumberFormat formatter = NumberFormat.simpleCurrency(
    locale: Localizations.localeOf(context).toString());

  return products.map((product) {
    return Card(
      clipBehavior: Clip.antiAlias,
      // TODO: Adjust card heights (103)
      child: Column(
        // TODO: Center items on the card (103)
        crossAxisAlignment: CrossAxisAlignment.start,
        children: <Widget>[
          AspectRatio(
            aspectRatio: 18 / 11,
            child: Image.asset(
              product.assetName,
              package: product.assetPackage,
              // TODO: Adjust the box size (102)
            ),
          ),
          Expanded(
            child: Padding(
              padding: const EdgeInsets.fromLTRB(16.0, 12.0, 16.0, 8.0),
              child: Column(
                // TODO: Align labels to the bottom and center (103)
                crossAxisAlignment: CrossAxisAlignment.start,
                // TODO: Change innermost Column (103)
                children: <Widget>[
                  // TODO: Handle overflowing labels (103)
                  Text(
                    product.name,
                    style: theme.textTheme.titleLarge,
                    maxLines: 1,
                  ),
                  const SizedBox(height: 8.0),
                  Text(
                    formatter.format(product.price),
                    style: theme.textTheme.titleSmall,
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
    ),
  ),
);

```



```

        ),
      ],
    ),
  );
}).toList();
}

```

注意：目前不會編譯及執行。我們還有一項變更。

如要設定文字樣式，我們會使用目前 **BuildContext** 的 **ThemeData**。

如要進一步瞭解文字樣式，請參閱 Material Guidelines 的「[字體排版](#)」一文。如要進一步瞭解主題設定，請在本程式碼研究室結束後，繼續學習 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題設定](#)。

此外，請先變更 `build()` 函式，將 **BuildContext** 傳遞至 `_buildGridCards()`，再嘗試編譯：

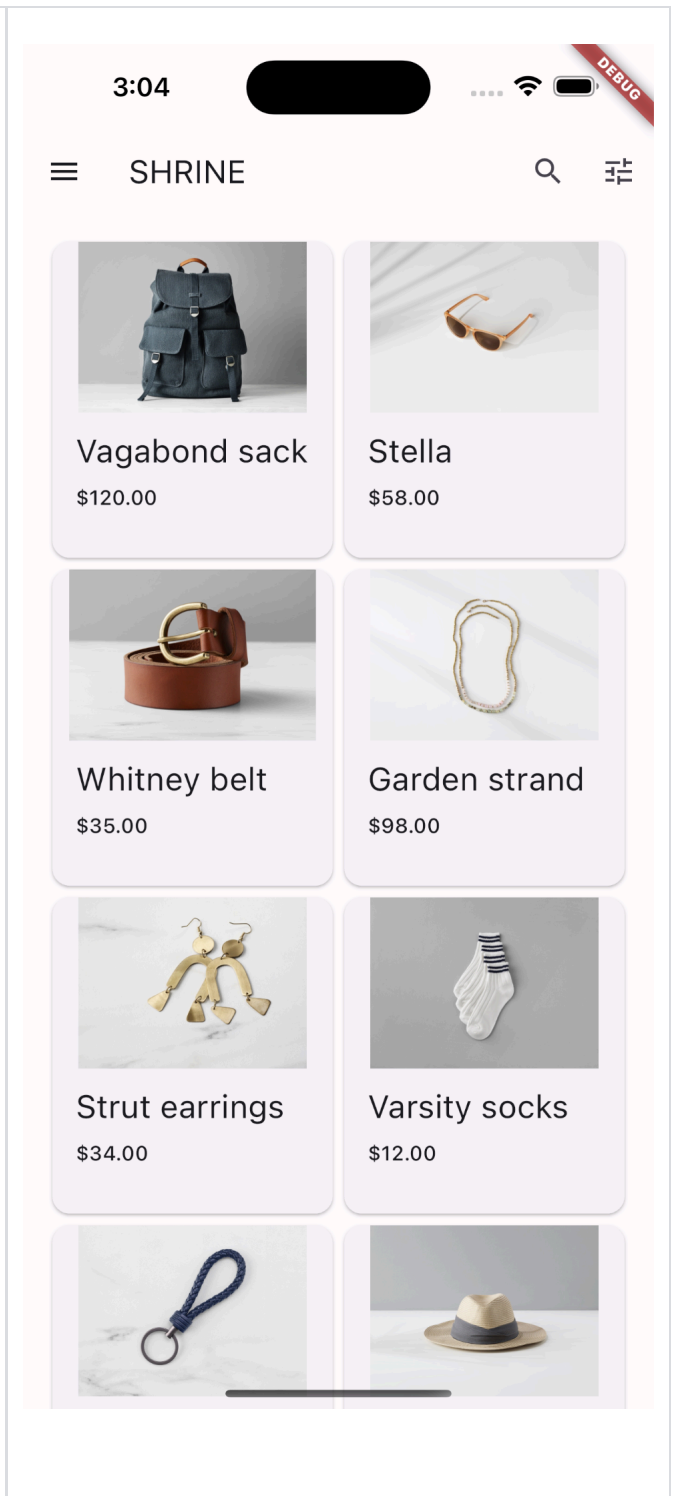
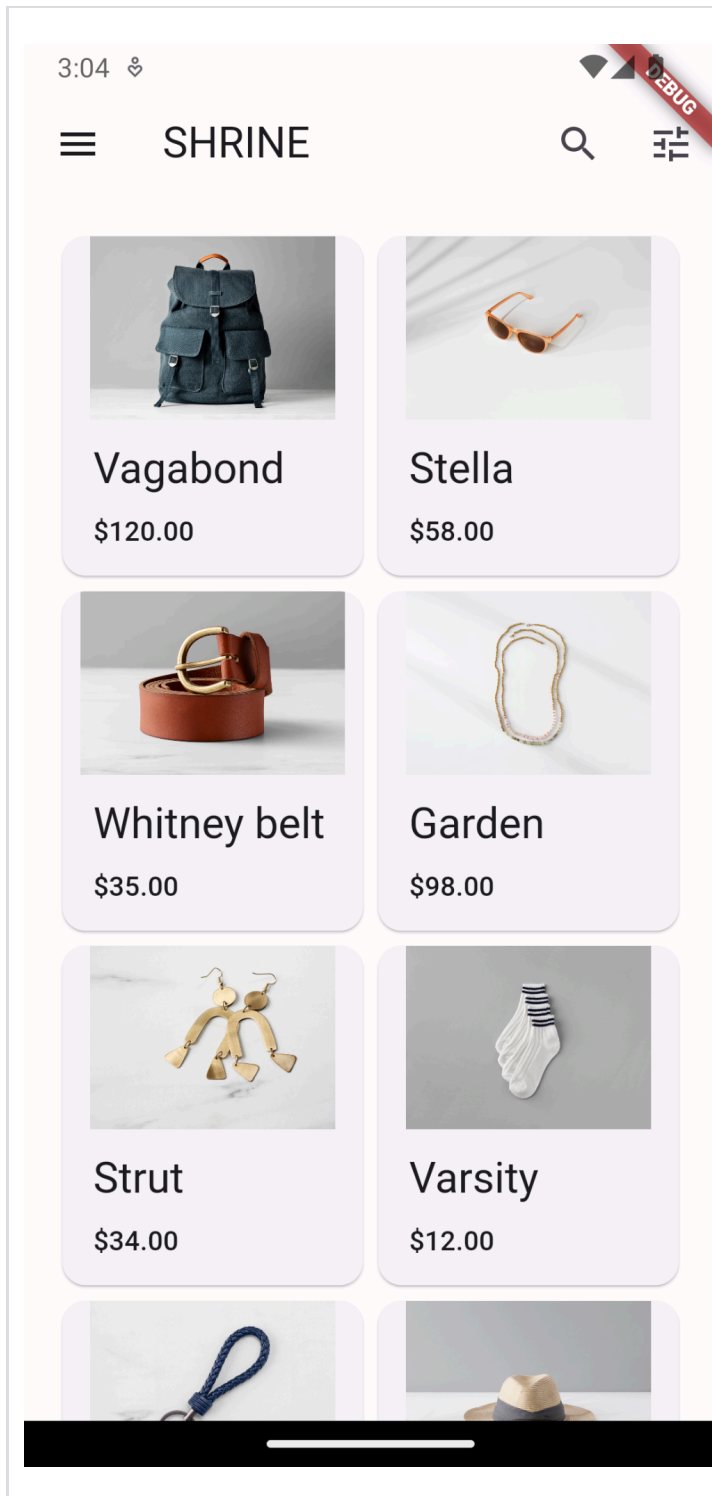
```

// TODO: Add a grid view (102)
body: GridView.count(
  crossAxisCount: 2,
  padding: const EdgeInsets.all(16.0),
  childAspectRatio: 8.0 / 9.0,
  children: _buildGridCards(context) // Changed code
),

```

熱重新啟動應用程式。

Android	iOS
---------	-----



您可能會發現我們並未在資訊卡之間加入任何垂直空間。這是因為這些元素預設會在頂端和底部有 4 點的邊界。

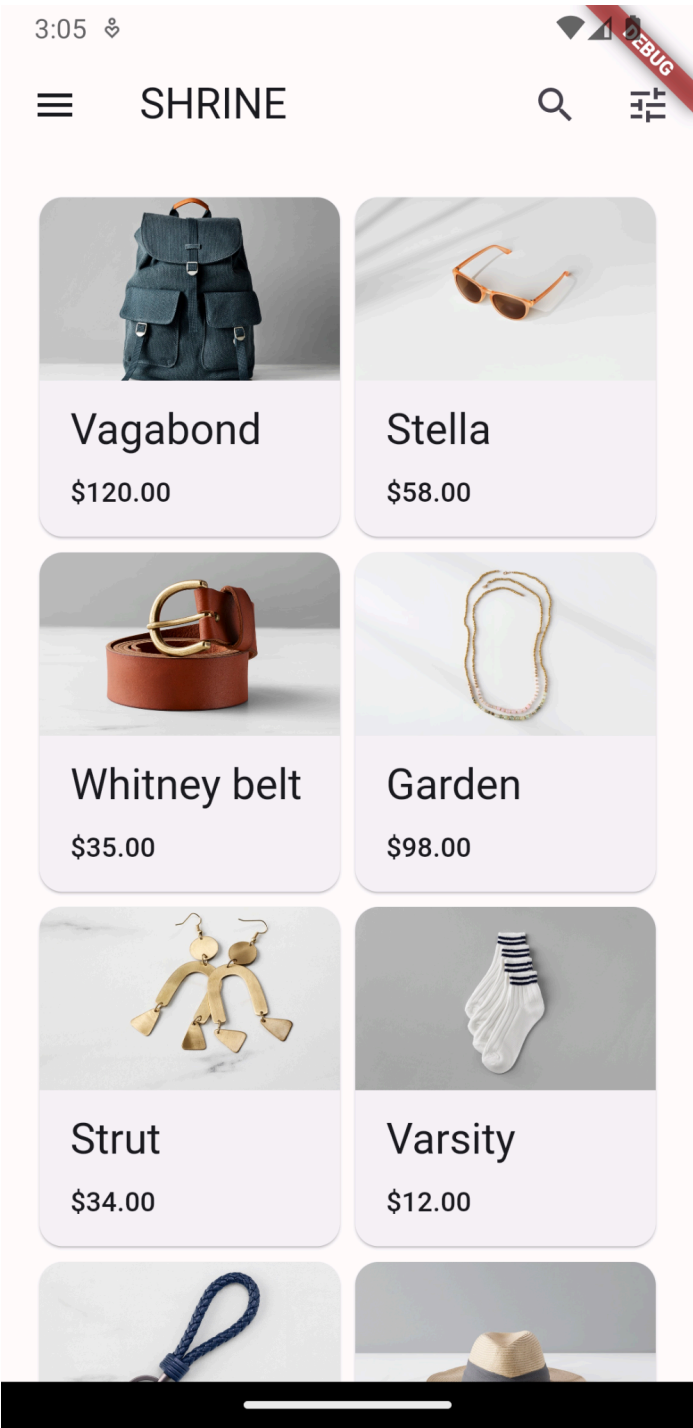
儲存專案。

產品資料會顯示，但圖片周圍有額外空間。圖片預設會以 `.scaleDown` 的 **BoxFit** 繪製 (在本例中)。我們將其變更為 `.fitWidth`，讓圖片稍微放大，並移除多餘的空白字元。

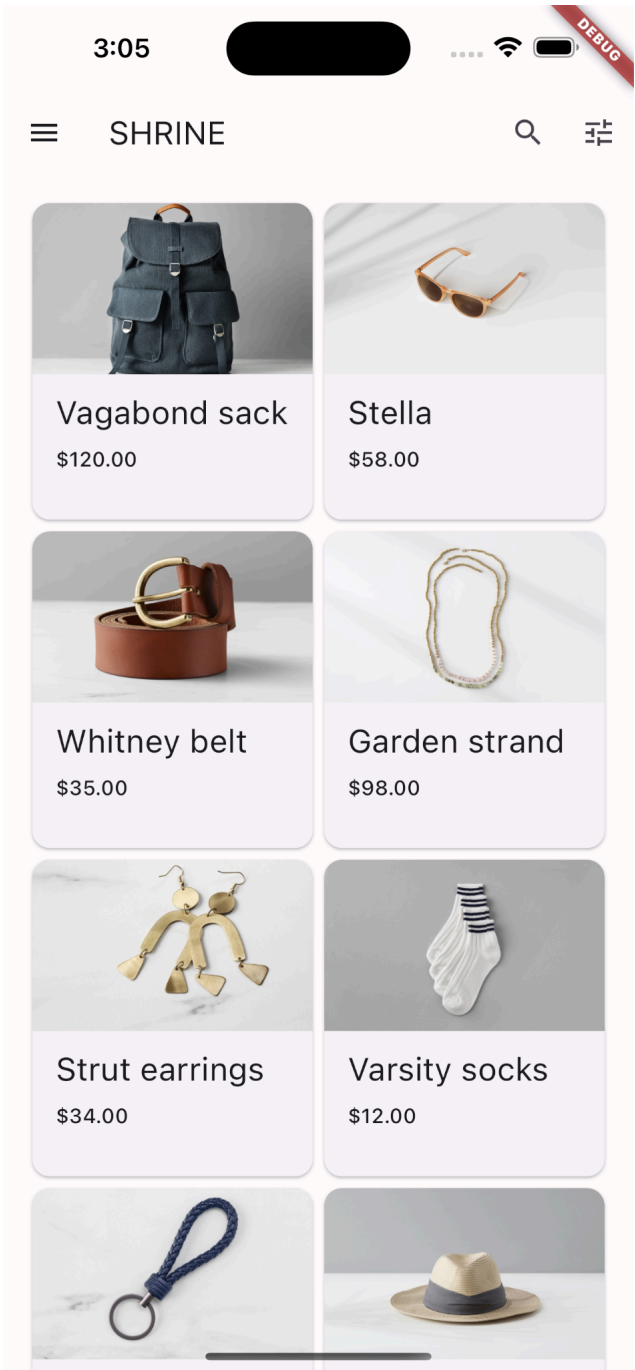
在圖片中新增 `fit:` 欄位，值為 `BoxFit.fitWidth`：

```
// TODO: Adjust the box size (102)
fit: BoxFit.fitWidth,
```

Android



iOS



我們的產品現在已完美顯示在應用程式中！

7. 恭喜！

我們的應用程式有基本流程，可將使用者從登入畫面帶往主畫面，方便他們查看產品。我們只用了幾行程式碼，就新增了頂端應用程式列 (包含標題和三個按鈕) 和資訊卡 (用於呈現應用程式內容)。現在的主畫面簡潔實用，基本架構和可執行的內容一目瞭然。

完成的 MDC-102 應用程式位於 `103-starter_and_102-complete` 分支中。

您可以根據該分支版本中的應用程式測試網頁版本。

後續步驟

我們已使用 Material Flutter 程式庫中的四個核心元件：上方應用程式列、資訊卡、文字欄位和按鈕！如要瞭解詳情，請前往 [Material 元件小工具目錄](#)。

雖然應用程式可正常運作，但尚未展現任何特定品牌或觀點。在 [MDC-103：利用顏色、形狀、高度和類型建立 Material Design 主題](#) 中，我們會自訂這些元件的樣式，展現充滿活力、現代感的品牌。
